

从 KL 的方向看 SFT 与 RL：大模型到底是在“学会做”，还是在“学会选”？

青稞AI 2026年5月23日 00:00 陕西

主页: <http://qingkeai.online/>

作者: sunnyzhao

原文: <https://zhuanlan.zhihu.com/p/2039859831348130507>

最近在看RL相关的几篇论文，发现和KL关联紧密，然后就深入看了下KL，Reverse KL与SFT，RL的对应关系，结合与ChatGPT的交互思考，整理文章如下。

如果只看公式,KL散度很容易让人困惑。它明明长得像两个分布之间的距离,却又不满足对称性:

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$$

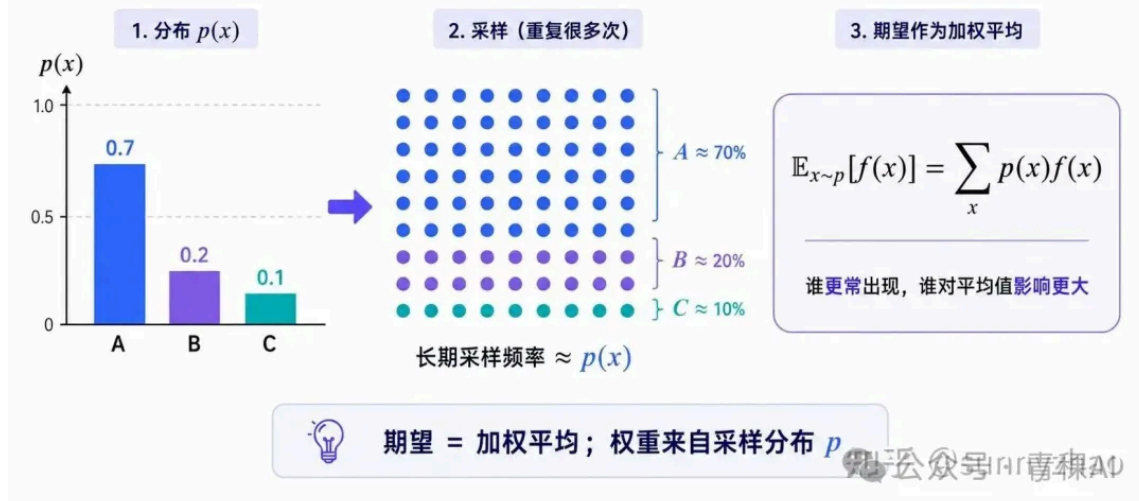
方向一换,训练行为就完全不同。

这件事放到大模型后训练里尤其关键。**SFT为什么常被理解成Forward KL? RL/RLHF 为什么又更接近Reverse KL?on-policy distillation 为什么和传统蒸馏不一样?** 这些问题背后,其实都在问同一件事:

期望到底是在谁的分布上取的?

也就是:样本从哪里来?轨迹由谁产生?模型是在解释别人的答案,还是在修正自己生成出来的答案?

为什么 $p(x)$ 能做权重?



要回答这几个问题,我们需要先从理解 KL 散度的方向性开始。

先理解 KL:方向来自“期望在哪个分布上取”

KL 散度的基本形式是:

$$D_{\text{KL}}(P \parallel Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

离散情况下展开就是:

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

这里最容易被忽略的是 $\mathbb{E}_{x \sim P}$ 。

它的意思不是随便比较 P 和 Q , 而是:

从 P 这个分布里采样 x , 然后计算 $\log(P(x)/Q(x))$ 的平均值。

所以 $P(x)$ 会出现在求和式前面, 成为每个位置的权重。

这不是因为 $P(x)$ 额外被赋予了权重身份, 而是因为样本本来就是从 P 中来的。某个 x 在 P 里越常出现, 它在长期采样平均中出现得就越多, 对最终 KL 的影响也就越大。

换句话说：

谁在期望符号下面,谁就是权重来源。

如果写的是 $D_{KL}(P \parallel Q)$,那就是在 P 上取期望,重点关注 P 会采样到的地方。

如果写的是 $D_{KL}(Q \parallel P)$,那就是在 Q 上取期望,重点关注 Q 会采样到的地方。

这就是 KL 非对称性的根源。

差异不在log本身,而在样本来自哪个分布。



Forward KL:目标有的,模型别漏掉

如果我们把 P 看成目标分布,把 Q 看成模型分布,那么：

$$D_{KL}(P \parallel Q)$$

就是**Forward KL**。

它问的问题是：

目标分布 P 会出现的地方,模型分布 Q 有没有覆盖到？

如果某个位置 $P(x) > 0$, 但 $Q(x) \rightarrow 0$, 那么惩罚会非常大。因为目标分布认为这个地方会出现, 但模型几乎不给概率。

所以 Forward KL 最害怕的是:

目标有, 模型没有。

这会带来一种典型训练倾向:

尽量覆盖目标分布里的所有模式。

如果目标分布里有 $A/B/C$ 三种合理答案模式, Forward KL 会倾向于让模型都学到。哪怕某些模式不够优雅、不够高奖励, 只要它们在数据分布里出现过, 模型就会被推动去覆盖。

所以 Forward KL 经常被称为:

- **mode-covering**
- **mass-covering**

如果用检索里的直觉类比, 它更像“重召回”。

它关心的是:

别漏掉目标分布里的东西。

但这个类比只是帮助理解, 不能严格等同于分类指标里的 recall。

Reverse KL: 模型生成的, 目标要认可

反过来看 Reverse KL:

$$D_{\text{KL}}(Q \parallel P)$$

如果 Q 是模型分布, P 是目标分布, 那么这就是 **Reverse KL**。

它问的问题变成:

模型自己会生成的地方, 目标分布 P 是否认可?

如果某个位置 $Q(x) > 0$, 但 $P(x) \rightarrow 0$, 那么惩罚会非常大。因为模型跑到了目标分布不支持的区域。

所以 Reverse KL 最害怕的是:

模型有, 目标不认可。

这会带来另一种训练倾向:

与其覆盖所有可能模式, 不如集中到少数更安全、更高价值、更被目标分布认可的模式上。

如果目标分布里有 $A/B/C$ 三个峰, Reverse KL 可能只选择其中一个最高、最稳、最容易获得奖励的峰, 而放弃其他模式。

所以 Reverse KL 经常被称为:

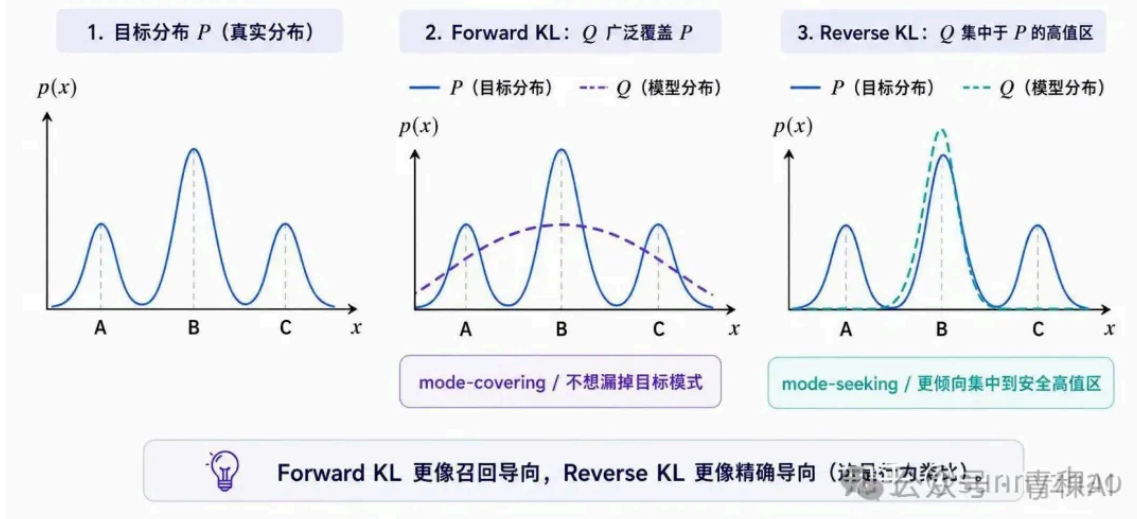
- **mode-seeking**
- **mode-selecting**

如果用检索里的直觉类比, 它更像“重精确”。

它关心的是:

模型生成出来的东西, 尽量都要落在目标认可的区域里。

Forward KL vs Reverse KL：覆盖与集中



SFT 为什么更接近 Forward KL

SFT的训练形式通常是：

给定输入 x 和人工标注答案 y , 让模型最大化 y 的概率。

也就是最小化：

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{y \sim p_{\text{data}}(y|x)} [\log \pi_{\theta}(y | x)]$$

这里的期望是在数据分布 p_{data} 上取的。

也就是说：

- 训练样本来自数据分布。
- 答案由人类标注或已有数据提供。
- 模型要做的是在这些答案上提高概率。

从 KL 角度看, SFT 可以近似理解为最小化：

$$D_{\text{KL}}(p_{\text{data}} \parallel \pi_{\theta})$$

也就是：

$$\text{KL}(\text{Target} \parallel \text{Model})$$

这里 Target 是数据分布,Model 是模型分布。

所以 SFT 的基本行为是：

数据里出现过的答案模式,模型尽量都要学到。

它更像模仿学习。

- 人怎么写,我怎么学。
- 老师给了什么答案,我就提升这些答案的概率。
- 数据分布覆盖了哪些表达方式,我就尽量拟合哪些表达方式。

这也是为什么SFT能让模型学会格式、任务、风格、指令跟随能力,却不一定能让模型在多个候选答案里总是选出最优的那个。

因为SFT的核心不是“选择最优”,而是“拟合示范”。

RL为什么更接近 Reverse KL

RL的结构与SFT不一样。

它不是只拿人工答案让模型照着学,而是让模型自己生成,然后通过 reward、偏好模型、verifier 或环境反馈来评价生成结果。

一个常见的KL正则化 RLHF 目标可以写成:

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta D_{\text{KL}}(\pi \parallel \pi_{\text{ref}})$$

这里最重要的是 $\mathbb{E}_{y \sim \pi}$, 也就是期望在当前模型策略 π 上取。

- 模型先生成。
- 奖励再评价。
- 策略再更新。

进一步看, 这个目标可以等价理解成模型在靠近一个reward诱导出来的目标分布:

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

于是优化目标可以写成近似:

$$\min_{\pi} D_{\text{KL}}(\pi \parallel \pi^*)$$

这就是 Reverse KL 的形式:

$$\text{KL}(\text{Model} \parallel \text{Target})$$

其中Model是当前策略 π , Target是由reward和reference policy共同诱导出来的理想分布 π^* 。

所以RL/RLHF的核心行为是:

模型先暴露自己的生成分布, 再根据奖励信号把概率质量推向更高价值的区域。

这和 SFT 的行为很不一样。

SFT 是:

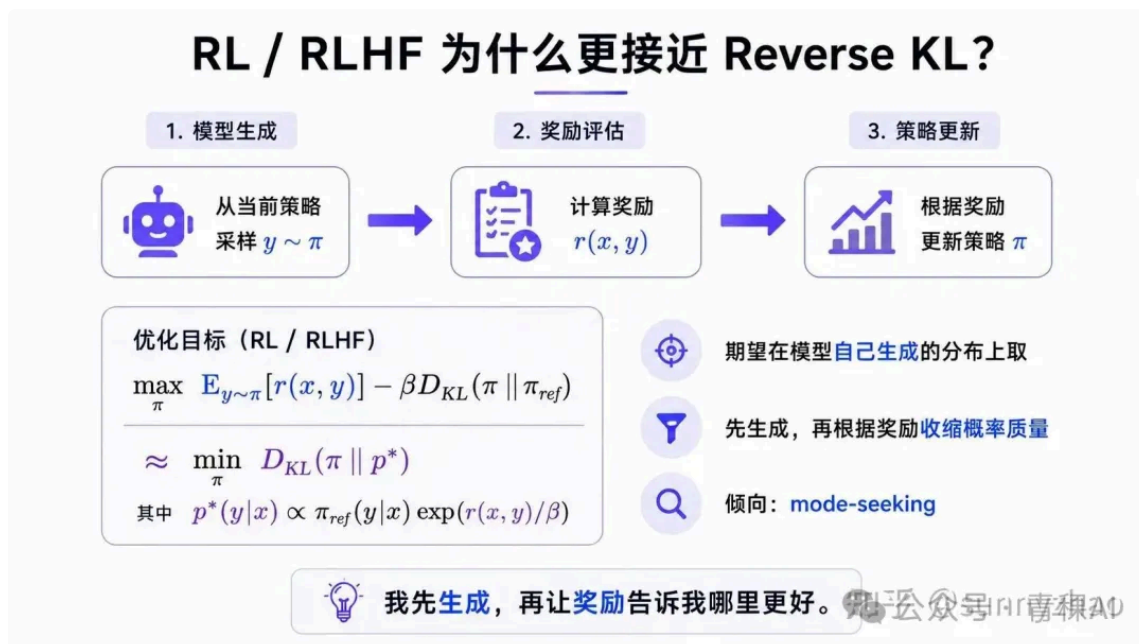
人类答案已经在那里, 模型去拟合。

RL 是:

模型自己先生成候选,再根据奖励筛选、强化、压缩。

因此 RL 更像”学会选”。

在多个可能答案里,哪些更有帮助,哪些更安全,哪些更符合偏好,哪些更能拿到高奖励,模型会逐渐把概率集中到这些区域。



传统蒸馏、on-policy distillation 与 KL 方向

因为最近on-policy distillation比较火, 此处做一点延伸思考, 这也是最近在看一篇OPD论文时所想到的:

KL 和 Reverse KL 好像和传统蒸馏、on-policy distillation 对应上了。一个是去 target 上采样,以 target 为轨迹标准。一个是去 model 上采样,以 model 为轨迹标准。

这个直觉大方向是对的,但”以 model 为轨迹标准”这句话描述不太精确。

更准确地说:

传统蒸馏 / SFT

在 *teacher* / *data* / *target* 分布上取样。

老师生成轨迹,或者数据提供轨迹。学生在这些轨迹上学习老师的分布。这是 **off-policy 于学生** 的,因为训练轨迹不是学生自己走出来的。

on-policy distillation / RL

在 *student/model/policy* 分布上取样。

学生自己生成轨迹,把自己真实会遇到的状态、错误、偏差暴露出来。老师、奖励模型或 verifier 再对这些轨迹给反馈。这是 **on-policy 于学生** 的,因为训练轨迹来自学生当前策略。

但需要注意:

model 采样,不等于 model 是标准。

在 on-policy distillation 里,学生生成的轨迹只是“训练发生的位置”,不是“正确答案的来源”。

真正的标准仍然来自 teacher。

- 学生走到哪里,老师就在哪里给反馈。
- 学生生成什么,老师就在这些生成内容上给出 logits、偏好、纠错信号或奖励。

所以更准确的表达是:

- **传统蒸馏**:老师先走,学生学老师走过的路。
- **on-policy distillation**:学生先走,老师在学生真实走到的位置上纠偏。
- **RL / RLHF**:模型先生成,奖励告诉模型哪些生成更值得保留。

这样对应之后也会发现 KL 真的无处不在,与 RL 有着紧密关联。而在后两者 RL 中,model 只是轨迹来源,不是评价标准。

支持集错配:为什么方向会改变惩罚对象

KL 的方向之所以重要,是因为它在支持集错配时惩罚对象完全不同。

看 Forward KL $D_{KL}(P \parallel Q)$ 。

如果 $P(x) > 0$, 但 $Q(x) = 0$, 那么:

$$P(x) \log \frac{P(x)}{0} = +\infty$$

这说明:

目标分布认为 x 会出现, 但模型完全不给概率。

Forward KL 会严重惩罚这种情况。所以它怕漏掉目标模式。

但如果 $P(x) = 0, Q(x) > 0$, 那么这一项是:

$$0 \cdot \log \frac{0}{Q(x)} = 0$$

按照极限约定, 这一项不产生惩罚。因为 P 根本不会采样到这个地方。

再看 Reverse KL $D_{\text{KL}}(Q \parallel P)$

如果 $Q(x) > 0$, 但 $P(x) = 0$, 那么:

$$Q(x) \log \frac{Q(x)}{0} = +\infty$$

这说明:

模型会生成 x , 但目标分布完全不认可。

Reverse KL 会严重惩罚这种情况。所以它怕模型跑到目标分布之外。

这就是两种 KL 行为差异的数学根源。

- **Forward KL**: 怕目标有、模型没有。
- **Reverse KL**: 怕模型有、目标不认。

为什么这会影响大模型后训练

理解这件事后,就能更清楚地看 SFT 和 RL 在后训练里的分工。

SFT 主要解决的是:

- 模型会不会按照人类示范去做。
- 模型能不能学会任务格式。
- 模型能不能覆盖训练数据中的回答方式。
- 模型能不能建立基本的指令跟随能力。

所以SFT更像打底。

它把模型从预训练的续写器,拉到一个能理解指令、能按格式回答、能模仿人类示范的状态。

但 SFT 的问题也很明显。

- 如果数据里有噪声,模型会学噪声。
- 如果数据里有平庸答案,模型会学平庸答案。
- 如果数据里多种风格混杂,模型会把这些风格都吸收进去。
- 如果任务需要在多个答案中选优,SFT本身不一定提供足够强的选择压力。

RL/RLHF主要解决的是:

- 模型在多个可能答案中,能不能更偏向高质量答案。

- 模型能不能减少明显不符合偏好的输出。
- 模型能不能把概率质量集中到更安全、更有用、更符合人类偏好的区域。
- 模型能不能从“会做”进一步走向“更会选”。

所以 RL 更像偏好对齐和行为压缩。

它不是简单扩充能力,而是在已有能力空间里改变概率分布,把更好的行为提上来,把低奖励行为压下去。

这也是为什么可以说：

SFT 让模型学会做。RL 让模型更会选。

更细一点说：

- SFT 学的是 demonstration, RL 优化的是 preference / reward。
- SFT 追求覆盖, RL 追求选择。
- SFT 更像 imitation, RL 更像 optimization。

不是 RL 都简单等同于 Reverse KL

这里还需要加一个边界。

“RL 对应 Reverse KL”这句话不是说所有传统 RL 天然都是 Reverse KL。

相对更准确的说法是:

在 KL-regularized RLHF、最大熵 RL、带 reference policy 的策略优化框架中, RL 目标可以被改写或理解成对某个 reward-induced target distribution 的 Reverse KL 最小化。

也就是:

$$\max_{\pi} \mathbb{E}_{y \sim \pi} [r(x, y)] - \beta D_{\text{KL}}(\pi \parallel \pi_{\text{ref}})$$

可以对应到:

$$\min_{\pi} D_{\text{KL}}(\pi \parallel \pi^*)$$

其中:

$$\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

这时说 RL 更接近 Reverse KL 是准确的。

但如果是一般传统 RL, 只写成最大化累计奖励:

$$\max_{\pi} \mathbb{E}_{\tau \sim \pi} [R(\tau)]$$

那它当然也是在当前策略上采样轨迹, 但不一定天然就是 Reverse KL 形式。只有当你引入熵正则、KL 正则、reference policy 或 reward-induced distribution 的解释时, Reverse KL 的结构才会显现出来。

同样, on-policy distillation 也不能简单说一定就是 Reverse KL。

因为有些方法虽然轨迹来自 student, 但在每个 student-generated context 上, 仍然可能用 teacher-to-student 的 token-level KL:

$$D_{\text{KL}}(\pi_{\text{teacher}} \parallel \pi_{\text{student}})$$

这种情况下, 它的“状态分布”是 on-policy 的, 但“动作分布上的 KL 方向”未必是 Reverse KL。

所以要区分两层：

第一层:轨迹从哪里来？

如果轨迹来自 $student / model$,那它是 on-policy。

第二层:KL 方向怎么写？

如果是 $KL(Model \parallel Teacher/Target)$,才是 Reverse KL。

如果是 $KL(Teacher/Target \parallel Model)$,仍然是 Forward KL,只是发生在 student 访问到的状态上。

这个区分很重要。否则容易把“on-policy”误解成“必然 reverse KL”。

更准确的表达是：

- on-policy 描述的是采样轨迹来自谁。
- Forward/Reverse KL描述的是分布比较的方向。两者虽然相关,但不是同一个概念。

KL 方向形成的统一理解

现在可以把整条线串起来。

KL 不是一个普通距离,而是一个有方向的信息代价。

方向来自：

期望在哪个分布上取。

Forward KL

$$D_{KL}(Target \parallel Model)$$

- 样本来自 Target。
- 模型要解释 Target。
- 目标有的,模型别漏。
- 行为倾向是 mode-covering。

对应到SFT/传统蒸馏,就是人类数据或老师输出作为学习轨迹,模型去拟合。

Reverse KL

$$D_{KL}(\text{Model} \parallel \text{Target})$$

- 样本来自 Model。
- 目标要评价 Model。
- 模型生成的,目标要认可。
- 行为倾向是 mode-seeking。

对应到KL正则化RL/RLHF,就是模型自己生成,再由奖励或目标分布筛选,把概率集中到高奖励区域。

传统蒸馏与on-policy distillation 的差别也可以从这里理解。

- **传统蒸馏**:teacher 生成,student 学。训练暴露的是 teacher 的轨迹。
- **on-policy distillation**:student生成,teacher评。训练暴露的是student自己会走到的轨迹。

对应一句话总结：

不是谁更像“真理”，而是谁在采样,谁在评价。

SFT/传统蒸馏里,target提供样本,model负责拟合。RL/on-policy 类方法里,model 提供样本,target/teacher/reward 负责评价。

这也就是从KL方向理解SFT、RL和on-policy distillation的关键。

结语

零零散散的思考了一堆，对KL和Reverse KL似乎也有了更深入的理解，用比较通俗的话来做下收尾。

Forward KL更像(模仿chatgpt-稳稳接住你)：

目标分布说：“我会出现的地方,你都要接住。”

Reverse KL更像：

目标分布说：“你自己想去的地方,必须是我认可的地方。”

SFT 更像:

人怎么写,我怎么学。

RL 更像:

我先生成,再让奖励告诉我哪里更好。

on-policy distillation 更像:

学生先暴露自己的轨迹,老师再针对这些轨迹给与细致反馈。

关键不是公式长得多复杂,而是那一个小小的期望符号:

$\mathbb{E}_{x \sim ?}$

谁在这里,谁就定义了训练看见的世界。

影响 OPD work 的因素有哪些？聊聊 Rethinking OPD 这一路

实录精选！全栈适配 DeepSeek V4，SGLang 如何实现系统级性能突破？

清华最新论文！动态 MoE 迁移算法 ZEDA：让 MoE 自己学会“摸鱼”，推理提速 20%

ACL 2026 | 多模态大模型是“看错”还是“想错”？浙大 VL-Calibration 让模型学会校准自信

加入青稞AI技术交流群

加入青稞AI技术交流群，不仅能与来自MIT、港中文、CMU、UCLA、斯坦福、清华、阿里、腾讯等名校名企AI研究员/开发者一起进行技术交流，同时还有一线青年AI研究员/开发者的Talk分享、青稞Tea、论文精读、招聘内推、国内外硕/博申请、大模型技术报告解读等。
备注：姓名+学校/公司+方向，暗号"AI"优先审核通过！



青稞AI

青稞 = 青年科学家，打造 AGI 时代的青年科学家社区
263篇原创内容

公众号

都看到这了，点个关注再走吧👉～

RL · 目录

上一篇 · 对比离线蒸馏！为什么选择 OPD？

